

自然语言处理 (2025 春季学期)

十、大语言模型的预训练



大纲

- 大语言模型的基本结构
- 注意力机制的优化
- 位置编码策略
- 长上下文处理策略
- 并行训练策略





大语言模型的基本结构

□ 传统自回归类型 – 以Llama为例

- Llama 由 Meta 于 2023 年 3 月发布
- 是 Decoder-only 单向语言模型，结构类似 GPT
- 开源引爆社区热情，衍生模型如 Alpaca、Vicuna 等迅速涌现

表 8-1 Llama、Llama 2、Llama 3 的模型大小、结构及训练超参数

模型名称	参数 /个	词表大小 /个	隐含层维数 /维	注意力头数 /个	层数 /层	训练词元数 /个	上下文长度	GQA
Llama	7B	32K	4,096	32	32	1.0T	2K	
	13B	32K	5,120	40	40	1.0T	2K	
	33B	32K	6,656	52	60	1.4T	2K	
	65B	32K	8,192	64	80	1.4T	2K	
Llama 2	7B	32K	4,096	32	32	2.0T	4K	
	13B	32K	5,120	40	40	2.0T	4K	
	34B	32K	6,656	52	60	2.0T	4K	✓
	70B	32K	8,192	64	80	2.0T	4K	✓
Llama 3	8B	128,256	4,096	32	32	15.0T+	8K	✓
	70B	128,256	8,192	64	80	15.0T+	8K	✓





大语言模型的基本结构

□ 传统自回归类型 – 以Llama为例

- 前置归一化：能够使模型的训练过程更加稳定

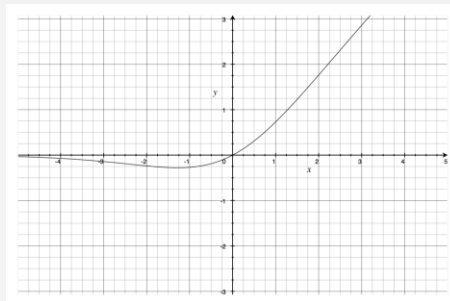
$$\overline{a_i} = \frac{a_i}{\text{RMS}(\mathbf{a})} g_i, \text{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n} \sum_{i=1}^n a_i^2}$$

- SwiGLU：是GLU的一种变体，是当前大模型中最常用的激活函数之一

$$\text{SwiGLU}_{\beta}(\mathbf{x}, \mathbf{W}, \mathbf{V}, \mathbf{b}, \mathbf{c}) = \text{Swish}_{\beta}(\mathbf{x}\mathbf{W} + \mathbf{b}) \otimes (\mathbf{x}\mathbf{V} + \mathbf{c})$$

$$\text{Swish}_{\beta}(\mathbf{x}) = \mathbf{x} \cdot \sigma(\beta \mathbf{x})$$

- RoPE：旋转位置编码（将在后续章节介绍）



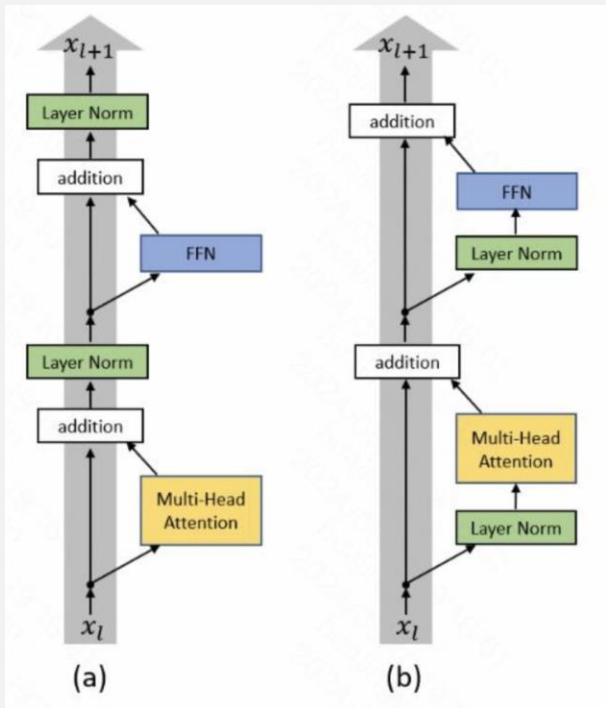


前置归一化

前置归一化 $x_{t+1} = x_t + F_t(\text{Norm}(x_t))$

后置归一化 $x_{t+1} = \text{Norm}(x_t + F_t(x_t))$

Pre Norm结构往往更容易训练，但最终效果通常不如 Post Norm



RMSNorm

$$\bar{a}_i = \frac{a_i}{\text{RMS}(\mathbf{a})} g_i, \quad \text{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n} \sum_{i=1}^n a_i^2}$$

LayerNorm

$$\bar{a}_i = \frac{a_i - \mu}{\sigma} g_i, \quad y_i = f(\bar{a}_i + b_i),$$

$$\mu = \frac{1}{n} \sum_{i=1}^n a_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (a_i - \mu)^2}.$$



大语言模型的基本结构

□ 混合专家模型类型 – 以Mixtral为例

- Mixture-of-Experts (MoE): 通过多个子模型（专家）协同处理任务
- 每个专家专注不同语言模式/结构/风格，提高多样性理解能力
- 在大规模多语言、多任务处理中表现突出

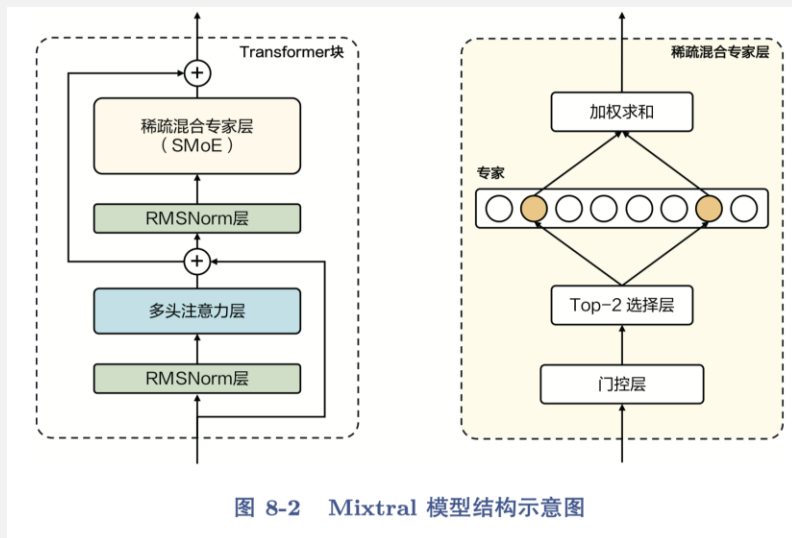


图 8-2 Mixtral 模型结构示意图

基于标准 Transformer 架构（Attention + FFN + Residual）
核心差异：引入门控层 + 多专家全连接层

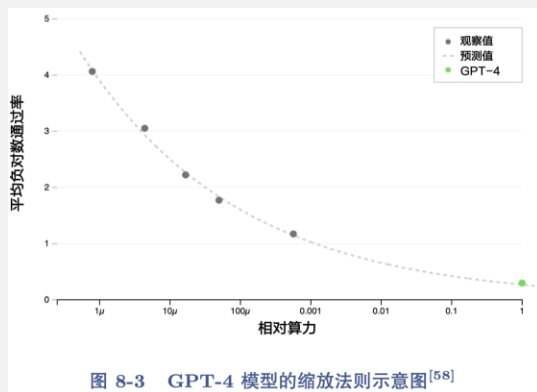
Token-level 动态专家选择：
每个词元根据上下文选择激活的 2 个专家
不是全局共享，而是词元粒度动态混合



大语言模型的基本结构

□ 缩放法则

- 描述模型大小、训练数据量、计算资源三者之间的相互关系与模型性能提升趋势
 - 模型越大 $\rightarrow$ 需要更多数据 & 更多计算 $\rightarrow$ 性能提升可预测
 - 是大语言模型设计与资源分配的理论指导依据
- 三要素之间关系： $C \approx 6ND$ （C:总计算量，N：模型参数量，D：训练词元数）





大语言模型的基本结构

□ 常见大模型对比

表 8-2 常见大语言模型对比

模型	发布时间	参数量/个	训练数据规模	训练设备
GPT-3	2020 年 6 月	175B	570 GB	1024 A100
GPT-3.5	2022 年 11 月	—	—	—
GPT-4	2023 年 3 月	—	—	—
Chinchilla	2022 年 3 月	70B	1.4T tokens	—
PaLM	2022 年 4 月	540B	—	6144 TPU v4
PaLM 2	2023 年 5 月	—	—	TPU v4
Llama	2023 年 2 月	7B, 13B, 33B, 65B	1T~1.4T tokens	2048 80G A100
Llama 2	2023 年 7 月	7B, 13B, 34B, 70B	2T tokens	2000 80G A100
Llama 3	2024 年 4 月	8B, 70B	15T tokens	24K H100
Falcon	2023 年 9 月	180B	3.5T tokens	4096 GPU
MPT	2023 年 5 月	7B, 30B	1T tokens	—
BLOOM	2022 年 7 月	176B	366B tokens	384 80G A100
BLOOMZ	2022 年 11 月	176B	—	—
OPT	2022 年 5 月	175B	180B tokens	992 80G A100
Galactica	2022 年 11 月	120B	106B tokens	—
ChatGLM	2023 年 5 月	6B	1T tokens	—
Qwen	2023 年 8 月	7B	2.2T tokens	—
Baichuan	2023 年 6 月	7B	1.2T tokens	—
Mistral	2023 年 9 月	7B	—	—
Mixtral	2023 年 12 月	8×7B	—	—
Mixtral	2024 年 4 月	8×22B	—	—
Phi	2023 年 9 月	1.3B	—	—
Gemma	2024 年 2 月	2B, 7B	2T~6T tokens	4096 TPU v5e





大纲

- 大语言模型的基本结构
- 注意力机制的优化
- 位置编码策略
- 长上下文处理策略
- 并行训练策略

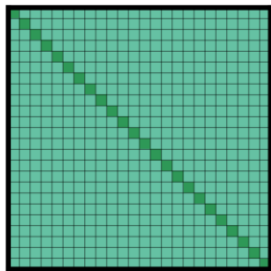




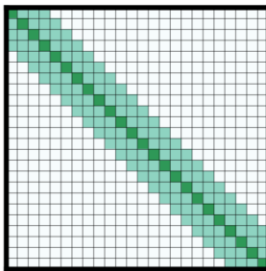
注意力机制的优化

□ 稀疏注意力 – Longformer为例

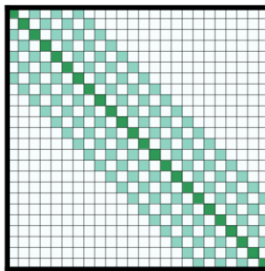
- 模型基于稀疏注意力机制，最大可处理长度扩展至4096
- 三种稀疏注意力模式
 - 滑动窗口注意力
 - 扩张滑动窗口注意力
 - 全局+滑动窗口注意力



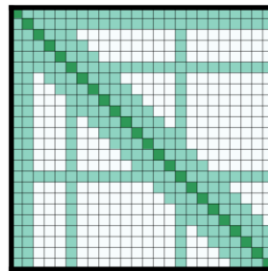
(a) 完整的注意力



(b) 滑动窗口注意力



(c) 扩张滑动窗口注意力



(d) 全局 + 滑动窗口

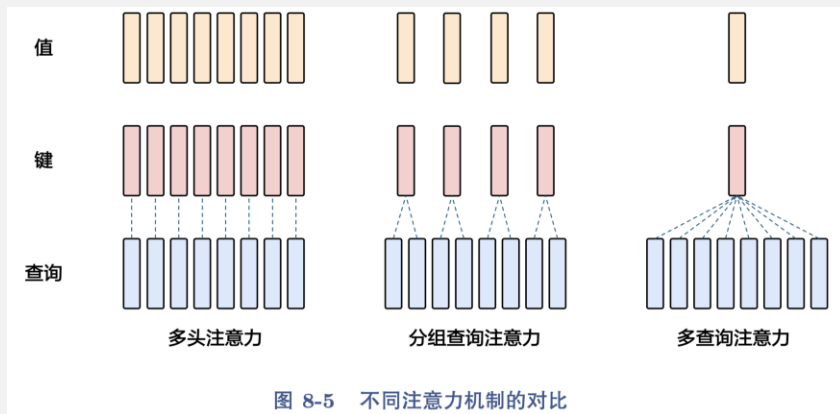




注意力机制的优化

□ 多查询注意力与分组查询注意力

- 背景：Transformer在训练阶段可并行处理全部输入，但在推理阶段每步需使用所有历史信息，导致高频加载键值张量、内存带宽压力大
- 多查询注意力：所有注意力头共享同一组键（K）与值（V）
- 分组查询注意力：多个 Query 头 共享一组 K/V 子集





注意力机制的优化

□ FlashAttention

— 背景

- 标准注意力机制频繁读写高带宽内存（HBM）
- Softmax、Dropout 等操作占用大量内存带宽，导致速度慢、能耗高

— 方法：将注意力重新分块计算，避免中间结果频繁写入 HBM

- 分块处理 + 多次遍历
- Softmax 归一化因子缓存，反向传播无需读取完整注意力矩阵

— 优势

- 更快的训练速度：FlashAttention 可以用更短的 Wall-clock 时间更快地训练 Transformer 模型
- 更好的模型效果：FlashAttention 使 Transformer 能够处理更长的序列，从而提高了模型的质量并带来新的功能
- 注意力基准测试：FlashAttention 在长度为 128 到 2K 的序列上比标准的注意力实现快3倍，并且长度可以扩展至64K



注意力机制的优化

FlashAttention

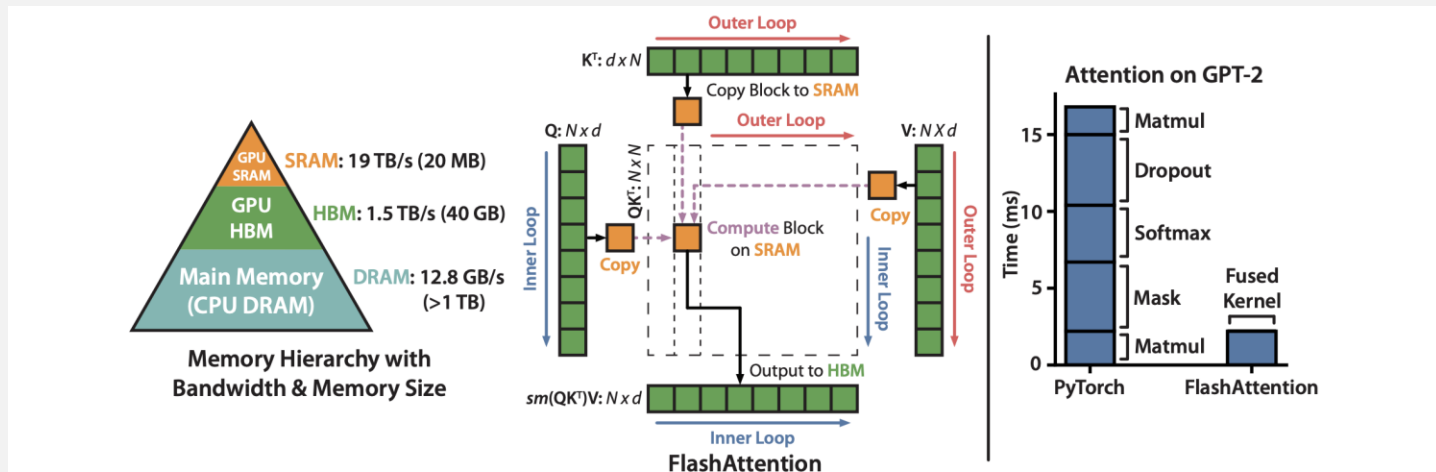


Figure 1: **Left:** FLASHATTENTION uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FLASHATTENTION loops through blocks of the K and V matrices and loads them to fast on-chip SRAM. In each block, FLASHATTENTION loops over blocks of Q matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FLASHATTENTION does not read and write the large $N \times N$ attention matrix to HBM, resulting in an $7.6\times$ speedup on the attention computation.



注意力机制的优化

□ FlashAttention

Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ where N is the sequence length and d is the head dimension, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where softmax is applied row-wise.

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM, which takes $O(N^2)$ memory. Often $N \gg d$ (e.g., for GPT2, $N = 1024$ and $d = 64$). We describe the standard attention implementation in Algorithm 0. As some or most of the operations are memory-bound (e.g., softmax), the large number of memory accesses translates to slow wall-clock time.

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-



注意力机制的优化

□ FlashAttention

数值稳定版本的 Softmax

$$m(x) := \max_i x_i, \quad f(x) := [e^{x_1 - m(x)} \quad \dots \quad e^{x_B - m(x)}], \quad \ell(x) := \sum_i f(x)_i, \quad \text{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

分块计算的 Softmax

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = [x^{(1)} \ x^{(2)}] \in \mathbb{R}^{2B}$ as:

$$m(x) = m([x^{(1)} \ x^{(2)}]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \begin{bmatrix} e^{m(x^{(1)}) - m(x)} f(x^{(1)}) & e^{m(x^{(2)}) - m(x)} f(x^{(2)}) \end{bmatrix},$$
$$\ell(x) = \ell([x^{(1)} \ x^{(2)}]) = e^{m(x^{(1)}) - m(x)} \ell(x^{(1)}) + e^{m(x^{(2)}) - m(x)} \ell(x^{(2)}), \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}.$$





注意力机制的优化

□ FlashAttention

Implementation details: Kernel fusion. Tiling enables us to implement our algorithm in one CUDA kernel, loading input from HBM, performing all the computation steps (matrix multiply, softmax, optionally masking and dropout, matrix multiply), then write the result back to HBM (masking and dropout in Appendix B). This avoids repeatedly reading and writing of inputs and outputs from and to HBM.

算法 8-2 FlashAttention 实现

Input: 矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ (位于 HBM 中), on-chip SRAM 大小为 M

Output: 输出矩阵 $\mathbf{O}$

1. 设置块大小 $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$
2. 在 HBM 中初始化 $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\mathbf{l} = (0)_N \in \mathbb{R}^N$, $\mathbf{m} = (-\infty)_N \in \mathbb{R}^N$
3. 将 $\mathbf{Q}$ 分为 $T_r = \lceil \frac{N}{B_r} \rceil$ 个块 $\mathbf{Q}_1, \mathbf{Q}_2, \dots, \mathbf{Q}_{T_r}$ 每个大小为 $B_r \times d$; 将 $\mathbf{K}, \mathbf{V}$ 分为 $T_c = \lceil \frac{N}{B_c} \rceil$ 个块 $\mathbf{K}_1, \mathbf{K}_2, \dots, \mathbf{K}_{T_c}$ 和 $\mathbf{V}_1, \mathbf{V}_2, \dots, \mathbf{V}_{T_c}$, 每个大小为 $B_c \times d$.
4. 将 $\mathbf{O}$ 分为 T_r 个块 $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$, 每个大小为 $B_r \times d$; 将 $\mathbf{l}$ 分为 T_r 个块 $\mathbf{l}_1, \dots, \mathbf{l}_{T_r}$, 每个大小为 B_r ; 将 $\mathbf{m}$ 分为 T_r 个块 $\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_{T_r}$, 每个大小为 B_r .
5. **for** $1 \leq j \leq T_c$ **do**
6. 将 $\mathbf{K}_j, \mathbf{V}_j$ 从 HBM 加载到 on-chip SRAM。
7. **for** $1 \leq i \leq T_r$ **do**
8. 将 $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{l}_i, \mathbf{m}_i$ 从 HBM 加载到 on-chip SRAM。
9. 在片上计算 $\mathbf{A}_{ij} = \mathbf{Q}_i \mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
10. 在片上计算 $\tilde{\mathbf{m}}_{ij} = \text{rowmax}(\mathbf{A}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{A}_{ij} - \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\mathbf{l}}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11. 在片上计算 $\mathbf{m}_i^{\text{new}} = \max(\mathbf{m}_i, \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B_r}$,
 $\mathbf{l}_i^{\text{new}} = e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{l}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \tilde{\mathbf{l}}_{ij} \in \mathbb{R}^{B_r}$.
12. 向 HBM 写入 $\mathbf{O}_i \leftarrow \text{diag}(\mathbf{l}_i^{\text{new}})^{-1} (\text{diag}(\mathbf{l}_i) e^{\mathbf{m}_i - \mathbf{m}_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$.
13. 向 HBM 写入 $\mathbf{l}_i \leftarrow \mathbf{l}_i^{\text{new}}$, $\mathbf{m}_i \leftarrow \mathbf{m}_i^{\text{new}}$.
14. **end**
15. **end**

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q} \mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
- 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
- 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P} \mathbf{V}$, write $\mathbf{O}$ to HBM.
- 4: Return $\mathbf{O}$.





大纲

- 大语言模型的基本结构
- 注意力机制的优化
- 位置编码策略
- 长上下文处理策略
- 并行训练策略



位置编码策略

□ RoPE (Rotary Positional Embeddings)

- 背景问题
 - 标准的绝对位置编码存在最大长度限制，难以适配超长序列或流式输入。
 - 对于长文本，传统位置编码难以捕捉 远距离相对位置信息。
- RoPE 的核心思想：将位置信息融入到注意力机制中的 Query 和 Key，通过二维旋转操作实现位置编码
- 具有以下优势：
 - 捕捉连续的相对位置信息，提升模型对长距离依赖的建模能力
 - 支持任意长度的输入，无需事先定义最大位置
 - 保持注意力得分的平移等变性（translation invariance）



位置编码策略

□ RoPE (Rotary Positional Embeddings)

- 在隐含层维度上，每两个维度划分为一组，那么总维度为 d 的向量可划分为 $d/2$ 个组
- 对每组赋予一个角度 θ ，并且记录其所对应的绝对位置 m ；
- 将每组中的两个元素旋转 $m\theta$ 角度，以此融入位置信息。

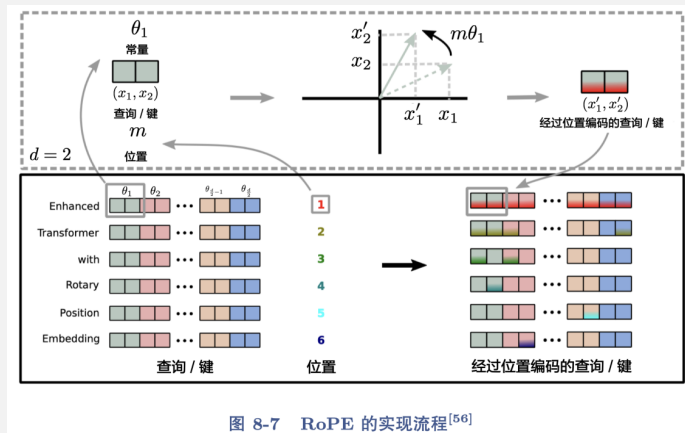


图 8-7 RoPE 的实现流程^[56]



位置编码策略

□ RoPE (Rotary Positional Embeddings)

- 在隐含层维度上，每两个维度划分为一组，那么总维度为 d 的向量可划分为 $d/2$ 组

对每

将每

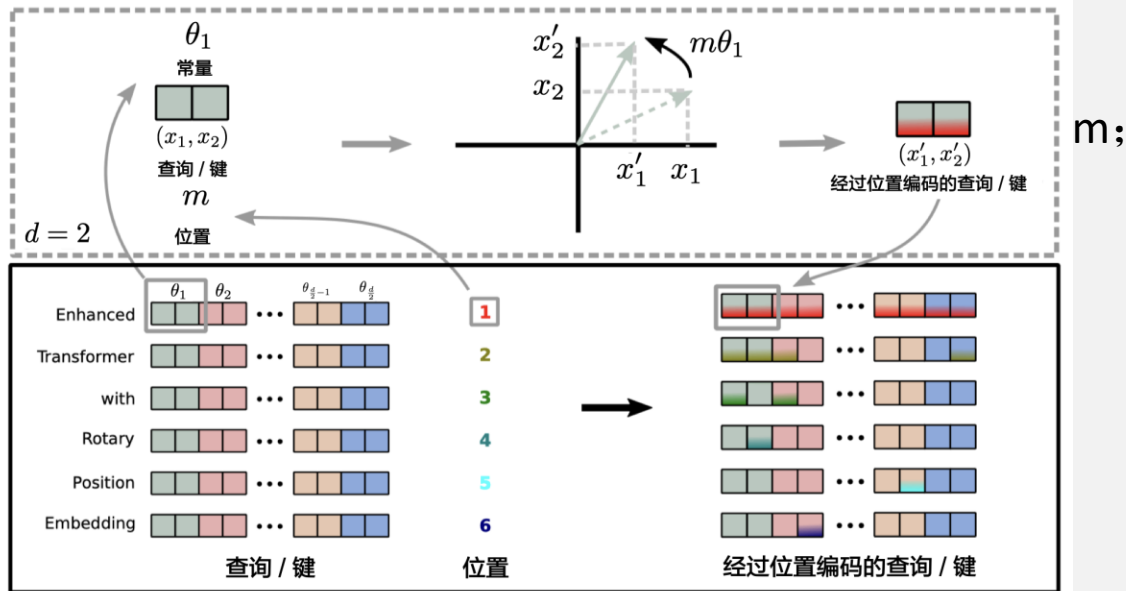


图 8-7 RoPE 的实现流程^[56]



位置编码策略

□ RoPE (Rotary Positional Embeddings)

- 在隐含层维度上，每两个维度划分为一组，那么总维度为 d 的向量可划分为 $d/2$ 个组
- 对每组赋予一个角度 θ ，并且记录其所对应的绝对位置 m ;
- 将每组中的两个元素旋转 $m\theta$ 角度，以此融入位置信息。

3.2.1 A 2D case

We begin with a simple case with a dimension $d = 2$. Under these settings, we make use of the geometric property of vectors on a 2D plane and its complex form to prove (refer Section (3.4.1) for more details) that a solution to our formulation Equation (11) is:

$$\begin{aligned} f_q(\mathbf{x}_m, m) &= (\mathbf{W}_q \mathbf{x}_m) e^{im\theta} \\ f_k(\mathbf{x}_n, n) &= (\mathbf{W}_k \mathbf{x}_n) e^{in\theta} \\ g(\mathbf{x}_m, \mathbf{x}_n, m - n) &= \text{Re}[(\mathbf{W}_q \mathbf{x}_m)(\mathbf{W}_k \mathbf{x}_n)^* e^{i(m-n)\theta}] \end{aligned} \quad (12)$$

where $\text{Re}[\cdot]$ is the real part of a complex number and $(\mathbf{W}_k \mathbf{x}_n)^*$ represents the conjugate complex number of $(\mathbf{W}_k \mathbf{x}_n)$. $\theta \in \mathbb{R}$ is a preset non-zero constant. We can further write $f_{\{q,k\}}$ in a multiplication matrix:

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix} \quad (13)$$





位置编码策略

□ RoPE (Rotary Positional Embeddings)

- 在隐含层维度上，每两个维度划分为一组，那么总维度为 d 的向量可划分为 $d/2$ 个组
- 对每组赋予一个角度 θ ，并且记录其所对应的绝对位置 m ;
- 将每组中的两个元素旋转 $m\theta$ 角度，以此融入位置信息。

3.2.2 General form

In order to generalize our results in 2D to any $\mathbf{x}_i \in \mathbb{R}^d$ where d is even, we divide the d -dimension space into $d/2$ sub-spaces and combine them in the merit of the linearity of the inner product, turning $f_{\{q,k\}}$ into:

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m \quad (14)$$

where

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \quad (15)$$

is the rotary matrix with pre-defined parameters $\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]\}$. A graphic illustration of RoPE is shown in Figure (1). Applying our RoPE to self-attention in Equation (2), we obtain:





位置编码策略

□ ALiBi (Attention with Linear Biases)

— 出发点

- 传统位置编码（如绝对编码）不适用于超长序列
- RoPE 虽然相对强大，但仍需在输入中显式加入位置结构
- ALiBi 直接在注意力打分中引入线性位置偏置项，简洁高效

— 特点与优势

- 不需要额外的位置嵌入参数
- 支持动态长度与超长序列
- 归纳偏好：自然偏向近期上下文，有利于结构理解
- 与 LLaMA 等主流大模型架构兼容





位置编码策略

□ ALiBi (Attention with Linear Biases)

- ALiBi 在注意力分数计算中加入位置差相关的线性惩罚项

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + m\mathbf{B}\right) \mathbf{V}$$

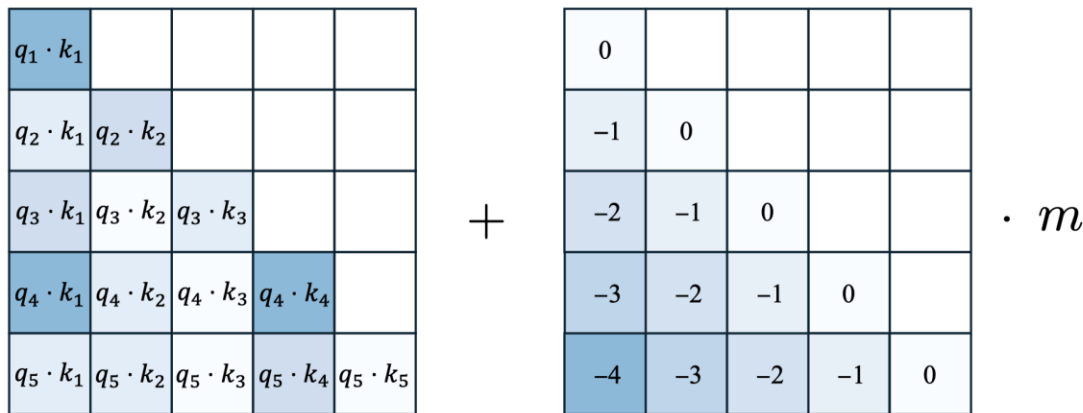


图 8-8 ALiBi 方法示意图



大纲

- 大语言模型的基本结构
- 注意力机制的优化
- 位置编码策略
- 长上下文处理策略
- 并行训练策略



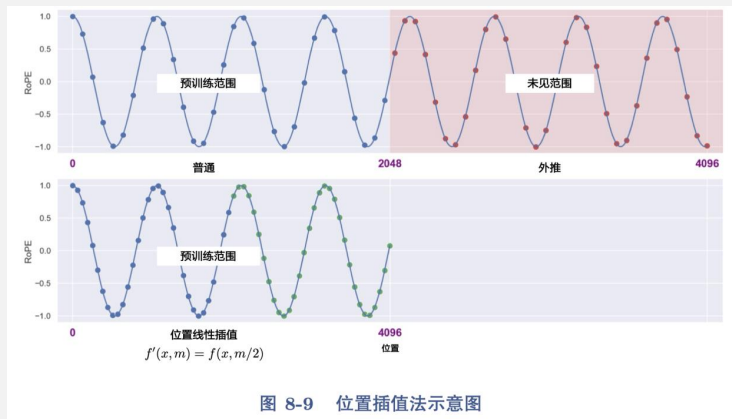


长上下文处理策略

□ 位置插值法

— 核心思想

- 将位置索引按比例缩放到原始窗口长度内
- 让新输入长度 L' 映射到模型预训练时支持的最大长度 L
- 缓解长上下文输入时的位置编码失效问题





长上下文处理策略

□ 位置插值法

— 核心思想

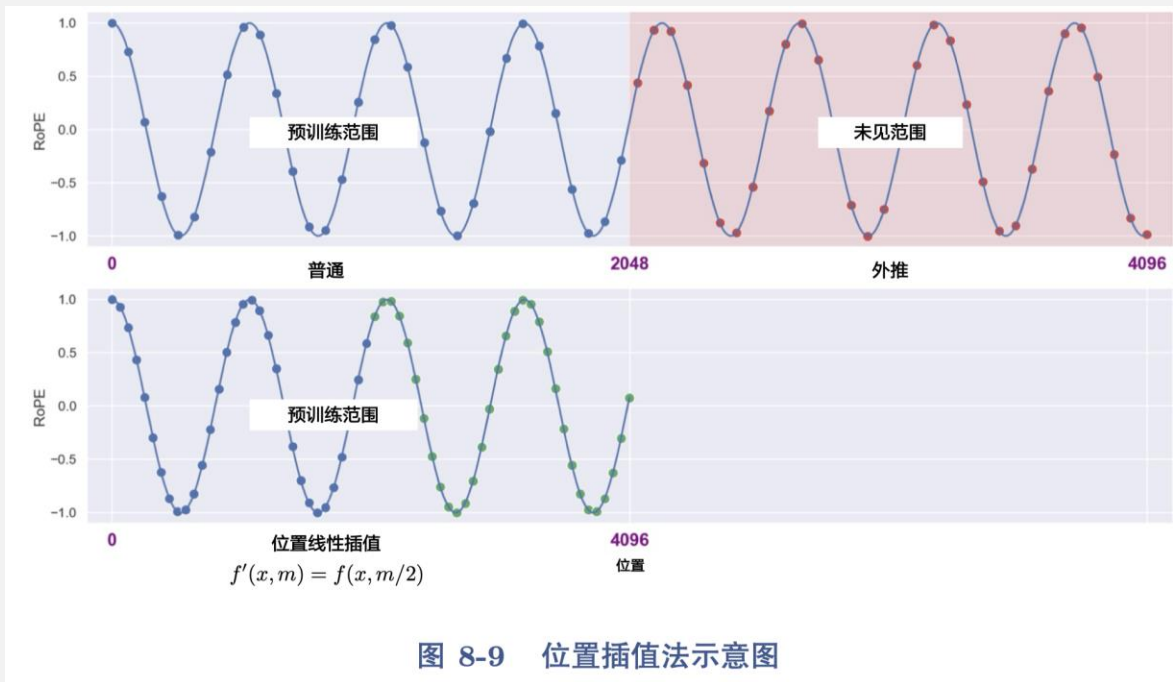


图 8-9 位置插值法示意图





长上下文处理策略

□ 基于NTK的方法

- 背景与动机
 - 外推法失效：长距离词元位置超出训练范围，导致注意力失准
 - 位置插值的不足：干扰了短距离词元的精确关系计算
- 解决思路：借助神经切线核（NTK）理论，平衡远距离与近距离信息
- RoPE中NTK改进公式：

$$b' = b \times \left(s_f \times \frac{L}{L_{\max}} - (s_f - 1) \right)^{\frac{d}{d-2}}$$



长上下文处理策略

□ LongLoRA

— 方法动机

- 标准 Transformer 的全局自注意力 $\rightarrow$ 计算复杂度 $O(n^2)$
- 大量远距离词元间的交互并非都重要
- 引入 Shift Short Attention (S^2 -Attn)：专注于局部且高效的注意力建模

— 注意力头分组

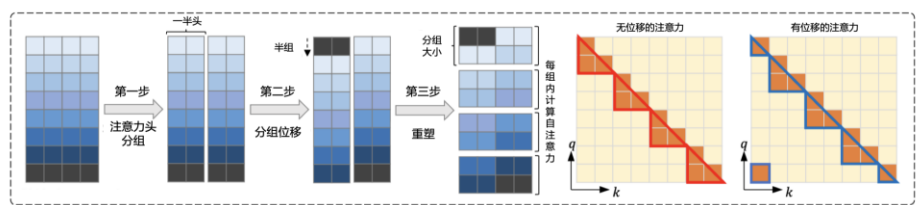
- 将长序列划分为多个固定大小的块（如 2048 tokens）；每个块独立进行注意力计算

— 分组位移

- 为促进块间交互，每组注意力头的一半将向前位移半个块

— 自注意力计算

- 每个 attention head 只在所在块内计算注意力；位移机制增强了跨组之间的信息交互





长上下文处理策略

□ YaRN (Yet another RoPE extension methods)

- 仅需1/10 的训练数据，1/4 的训练步数；效果优于位置插值法和标准 NTK
- 包含两大核心设计：NTK-by-parts 插值法；注意力温度调节机制
- NTK-by-parts 插值法
 - 传统插值法假设所有维度影响相同，忽略频率差异
 - 现实中不同维度对应不同 RoPE 波长 λ
- 核心策略：
 - 波长较短 ($\lambda \ll L$) $\rightarrow$ 不插值（保留局部精度）
 - 波长较长 ($\lambda \geq L$) $\rightarrow$ 插值（保留全局位置）
 - 中间波长 $\rightarrow$ 混合插值（渐变策略）



长上下文处理策略

□ YaRN (Yet another RoPE extension methods)

- 仅需1/10 的训练数据，1/4 的训练步数；效果优于位置插值法和标准 NTK
- 包含两大核心设计：NTK-by-parts 插值法；注意力温度调节机制
- 注意力温度调节机制
 - 在计算注意力时引入温度系数 t 以调节困惑度，且这种调节具有良好推广性，可适配到不同数据样本和扩展上下文长度后的词元位置。
- 核心策略：
 - 注意力计算公式转换为：
$$\text{softmax} \left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{t \sqrt{|D|}} \right)$$
 - 可在Q和K对应的RoPE上乘以缩放系数 $\sqrt{\frac{1}{t}}$ 以达到等效计算的目的
 - 对于Llama 和 Llama 2 模型，温度系数 t 通常取值：
$$\sqrt{\frac{1}{t}} = 0.1 \ln(s) + 1.$$





大纲

- 大语言模型的基本结构
- 注意力机制的优化
- 位置编码策略
- 长上下文处理策略
- 并行训练策略





并行训练策略

□ 数据并行

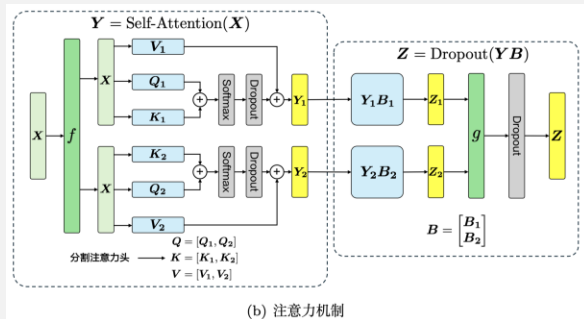
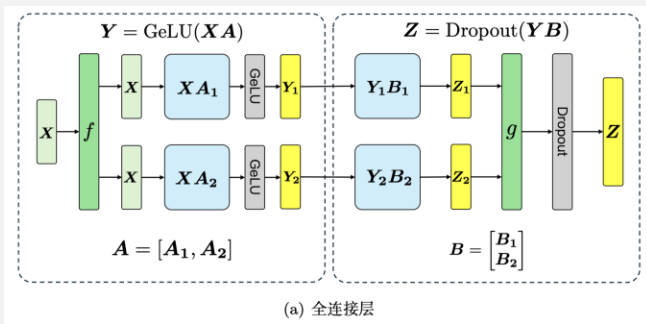
- 数据并行是一种分布式训练策略，每个计算单元（如 GPU）拥有模型的完整副本，并各自处理一部分数据，协同完成模型训练
- 关键步骤
 - 数据划分：将训练数据集平均分配到多个计算单元中处理
 - 并行计算：各 GPU 独立进行前向传播与反向传播
 - 梯度聚合：将所有计算单元的梯度结果进行平均
 - 权重同步：将更新后的模型参数广播回所有计算单元
- 优势
 - 显著加速训练过程；易于实现，主流框架如 PyTorch 和 TensorFlow 原生支持
 - 适用广泛，支持多种模型和任务类型
- 挑战
 - 通信开销大：尤其是在大模型中，梯度同步可能成为瓶颈
 - 显存压力大：每个 GPU 都需保存完整模型副本
 - 扩展性受限：随着设备数量增加，效率可能下降



并行训练策略

□ 模型并行

- 模型并行是一种将模型的不同部分分布在多个设备上进行训练的策略，适用于单台设备无法容纳完整模型的场景，尤其常用于超大语言模型的训练
- 工作流程
 - 模型划分：将模型按层或模块切分至不同设备
 - 分布式计算：前向传播中，输入逐层传递至不同 GPU；反向传播时，梯度按层回传。
 - 权重更新：各设备独立更新自己负责部分的参数
 - 通信协同：跨设备的数据传递需高效调度，确保计算与同步一致





并行训练策略

□ 流水线并行

- 流水线并行是一种将超大语言模型划分为多个计算阶段，并在多个设备上依次处理这些阶段的并行计算策略。灵感源自工业流水线，能够显著提升训练效率和资源利用率
- 工作机制
 - 模型分割：将模型按层或模块划分为若干阶段
 - 设备配置：根据各阶段的计算量分配至不同 GPU
 - 流水执行：多个微批次同时在不同阶段处理，实现阶段间并行
 - 同步更新：每轮迭代后聚合梯度，统一更新参数

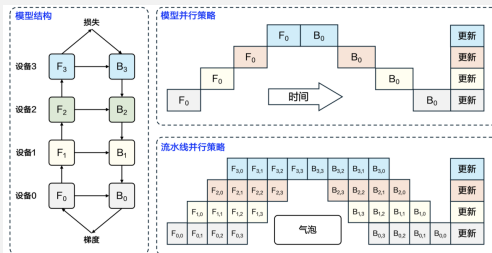


图 8-12 流水线并行策略 GPipe 流程示意图 (F: 前向传播; B: 反向传播)



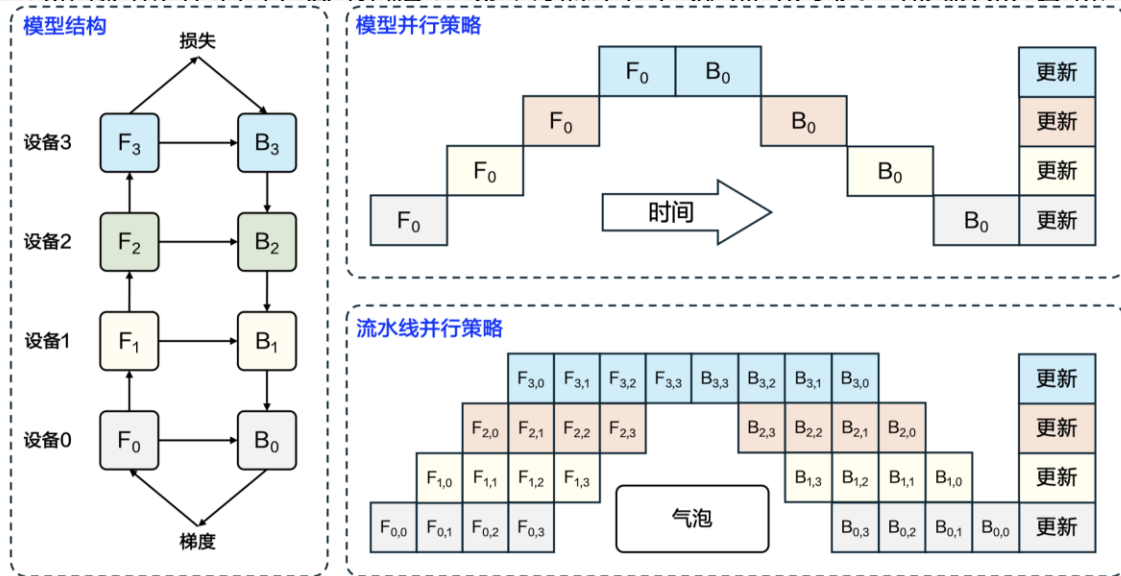
并行训练策略

□ 流水线并行

- 流水线并行是一种将超大语言模型划分为多个计算阶段，并在多个设备上依次处理这些阶段的并行计算策略。灵感源自工业流水线，能够显著提升训练效率和资源利用率。

工作

-
-
-
-





并行训练策略

□ 混合并行

- 混合并行是一种将数据并行、模型并行、流水线并行等多种并行策略灵活结合的分布式训练方法，专为超大规模语言模型设计
- 关键特点
 - 高性能：多策略协同，大幅提升训练效率与吞吐量
 - 高灵活性：可按模型结构与硬件特性自定义并行组合
 - 强扩展性：支持跨 GPU、跨节点的大规模训练任务
 - 资源优化：内存与计算资源可按阶段或维度精准分配
- 应用场景：在超大模型训练中，通常采用如下混合结构：
 - 张量并行（模型内部）：对单层进行切分（如 Attention/MLP）
 - 流水线并行（模型层级）：跨层级划分模块
 - 数据并行（批处理维度）：跨数据批次同步梯度
- 实现挑战：并行策略调度复杂；通信与同步代价高；需依赖高效框架（如 DeepSpeed、Megatron-LM、Colossal-AI）



并行训练策略

□ 零冗余优化（ZeRO）

- ZeRO是一种由微软提出的内存优化技术，专为超大规模模型训练设计，通过去除冗余副本显著降低 GPU 内存占用，从而支持更大模型的训练
- 核心策略
 - 参数分割：将模型参数切分后分布在不同 GPU 上，各自只保存部分参数
 - 梯度分割：各设备仅保存自己负责部分的梯度，减少反向传播内存需求
 - 优化器状态分割：如动量、方差等状态信息按需切分，进一步减轻负载
- 关键优势
 - 显著节省显存，支持更大的模型在有限硬件上训练
 - 提升计算效率，优化器状态与计算负担更均衡
 - 良好的可扩展性，支持跨多个 GPU/节点高效并行
- 应用场景
 - GPT-3、OPT、LLaMA 等超大语言模型训练
 - DeepSpeed、Colossal-AI 等训练框架的核心优化组件





并行训练策略

□ DeepSpeed

- DeepSpeed 是由微软研究院开发的高性能分布式训练库，专为超大规模语言模型的训练和部署设计，支持数十亿乃至上千亿参数的模型
- 核心能力
 - 并行训练支持：内置数据并行、层间/层内模型并行、流水线并行，适配多种规模和拓扑结构
 - 高效内存优化：集成 ZeRO 系列优化策略（ZeRO-1/2/3/++），显著降低显存需求
 - 高性能通信：通过混合精度训练、压缩通信等手段，减少跨设备数据传输开销
 - 强兼容性：无缝集成 PyTorch 与 Hugging Face transformers，可快速接入现有模型
 - 出色可扩展性：支持千卡级别的分布式训练，具备自动调优与弹性调度能力

项目	模型			
	ZeRO-1	ZeRO-2	ZeRO-3	ZeRO-Infinity
梯度分割	✓	✓	✓	—
优化器状态分割		✓	✓	—
模型参数分割			✓	—
内存优化效率	低	中	高	极高（利用 NVMe）
通信优化	低	中	高	高





小结

- 大语言模型的基本结构
- 注意力机制的优化
- 位置编码策略
- 长上下文处理策略
- 并行训练策略

